

Observability & Chaos Engineering

EMPOWERING High Performance
Technology Teams

Duration	Activity
10 mins	Welcome and Objectives Introduction to observability and chaos engineering concepts.
30 mins	Comprehensive Monitoring Log aggregation, metric collection, and distributed tracing for end-to-end system monitoring
30 mins	Multi-Layer Monitoring Architecture Insights across infrastructure, application, and business metrics.
10 mins	Break
30 mins	Monitoring Frameworks and Best Practices Advanced anomaly detection and alert management.
30 mins	Controlled Failure Injection Chaos experiments to test system resilience.
30 mins	Continuous Improvement Structured experiments to enhance reliability.
15 mins	Key Takeaways and Case Study Discussion Real-world examples and actionable insights.
10 mins	Q&A

Note: For sessions that are completely demo/Lab or notebook based, please keep one agenda slide and explain the same thing in the beginning so that participants are aware they do not have PPT.

Introduction to Observability and Chaos Engineering

1. Defining Observability

Observability is the ability to understand the internal state of a system by analysing logs, metrics, and traces.

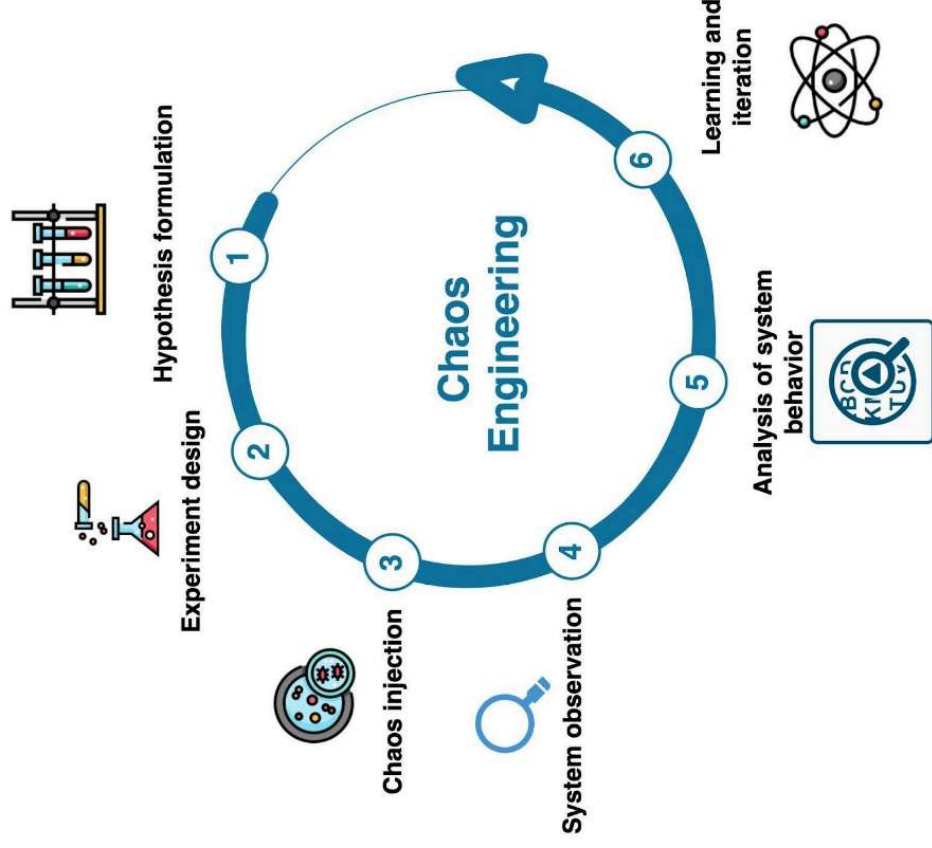
2. Defining Chaos Engineering

Chaos Engineering tests a system's resilience by intentionally injecting failures in controlled environments.

3. Objectives for the Session

- Implement end-to-end monitoring with observability tools.
- Design and execute chaos experiments to enhance system resilience.

Chaos Engineering

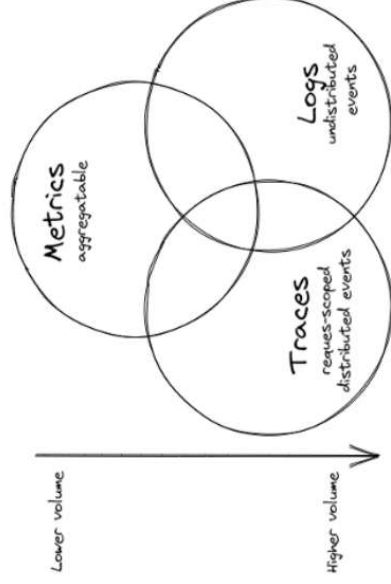


Log Aggregation, Metric Collection, and Distributed Tracing

1. Log Aggregation

- Collect logs from all services using centralized tools like the **ELK Stack (Elasticsearch, Logstash, Kibana)** or **Splunk**.
- Correlate logs to identify patterns across distributed systems.

Example: Debugging a payment processing error by tracking transaction IDs in logs.



2. Metric Collection

- Capture real-time metrics like CPU, memory usage, request latency, and error rates using **Prometheus** or **Datadog**.
- Visualize metrics with **Grafana** dashboards for actionable insights.

3. Distributed Tracing

- Use **Jaeger** or **Open Telemetry** to trace requests across microservices.
- Example:** Diagnose a slow API request by visualizing the call stack and identifying bottlenecks.

Multi-Layer Monitoring Architecture

1. Layers of Monitoring

- **Infrastructure Metrics:** CPU, disk I/O, memory usage.
- **Application Metrics:** Error rates, response times, throughput.
- **Business Metrics:** Transaction success rates, cart abandonment rates.

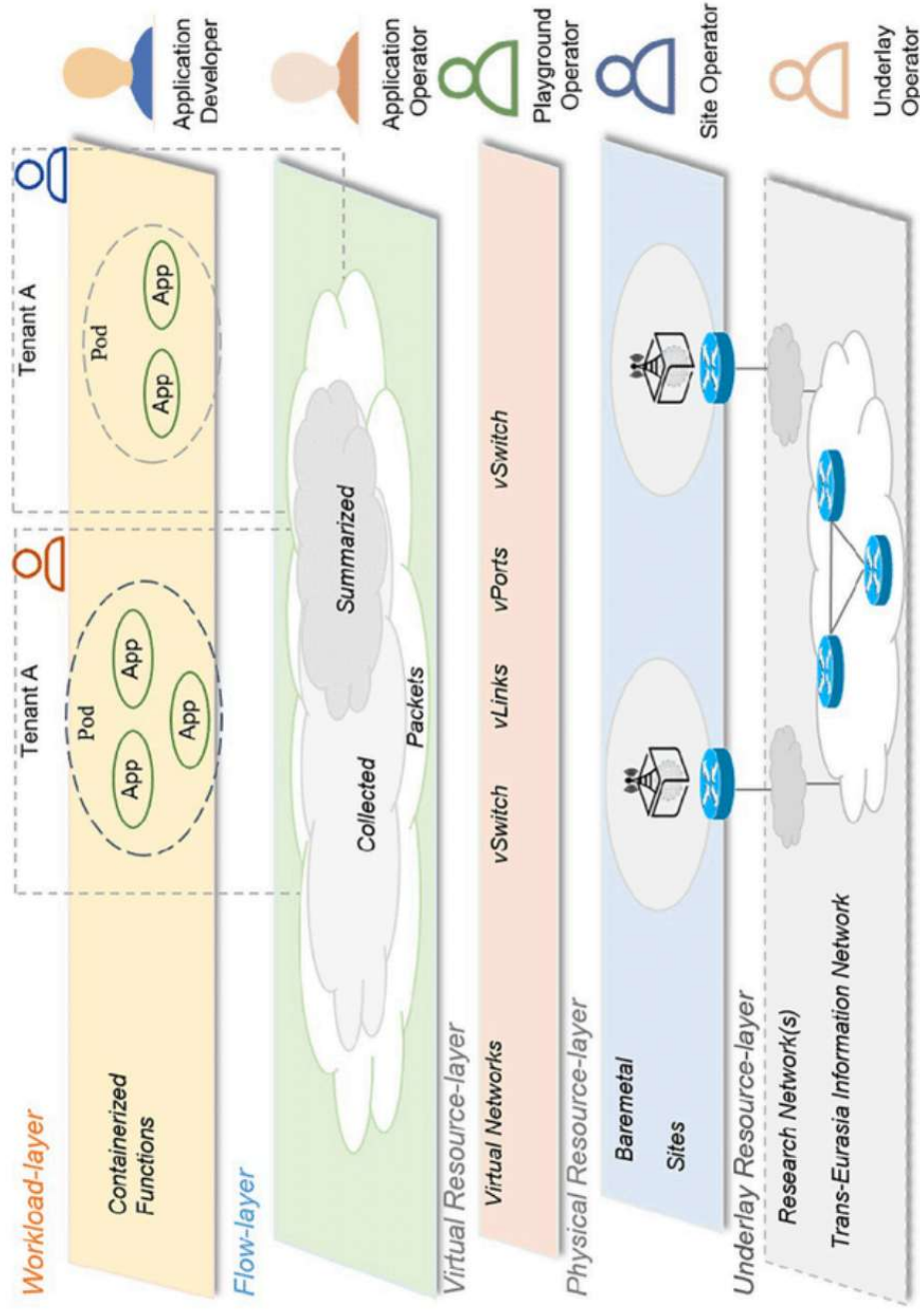
2. Unified Monitoring

- Correlate metrics across layers using tools like **Datadog** or **New Relic**.
- Create composite metrics to measure overall system health.

Example:

Build a Grafana dashboard visualizing metrics from all layers (infrastructure, application, and business).

Concept of multi layer visibility for unified monitoring



Monitoring Frameworks and Best Practices

1. Advanced Anomaly Detection

- Use ML-based tools like **Datadog Anomaly Detection** or **Dynatrace** to identify unusual patterns.
- **Example:** Detecting a 10% increase in error rates during a high-traffic event.

2. Alert Management

- Avoid alert fatigue by setting intelligent thresholds for alerts.
- Use escalation policies with **PagerDuty** or **OpsGenie** to prioritize responses.

Activity:

Configure an alerting system to detect latency spikes in a simulated environment.

Chaos Engineering

How Chaos Engineering Works

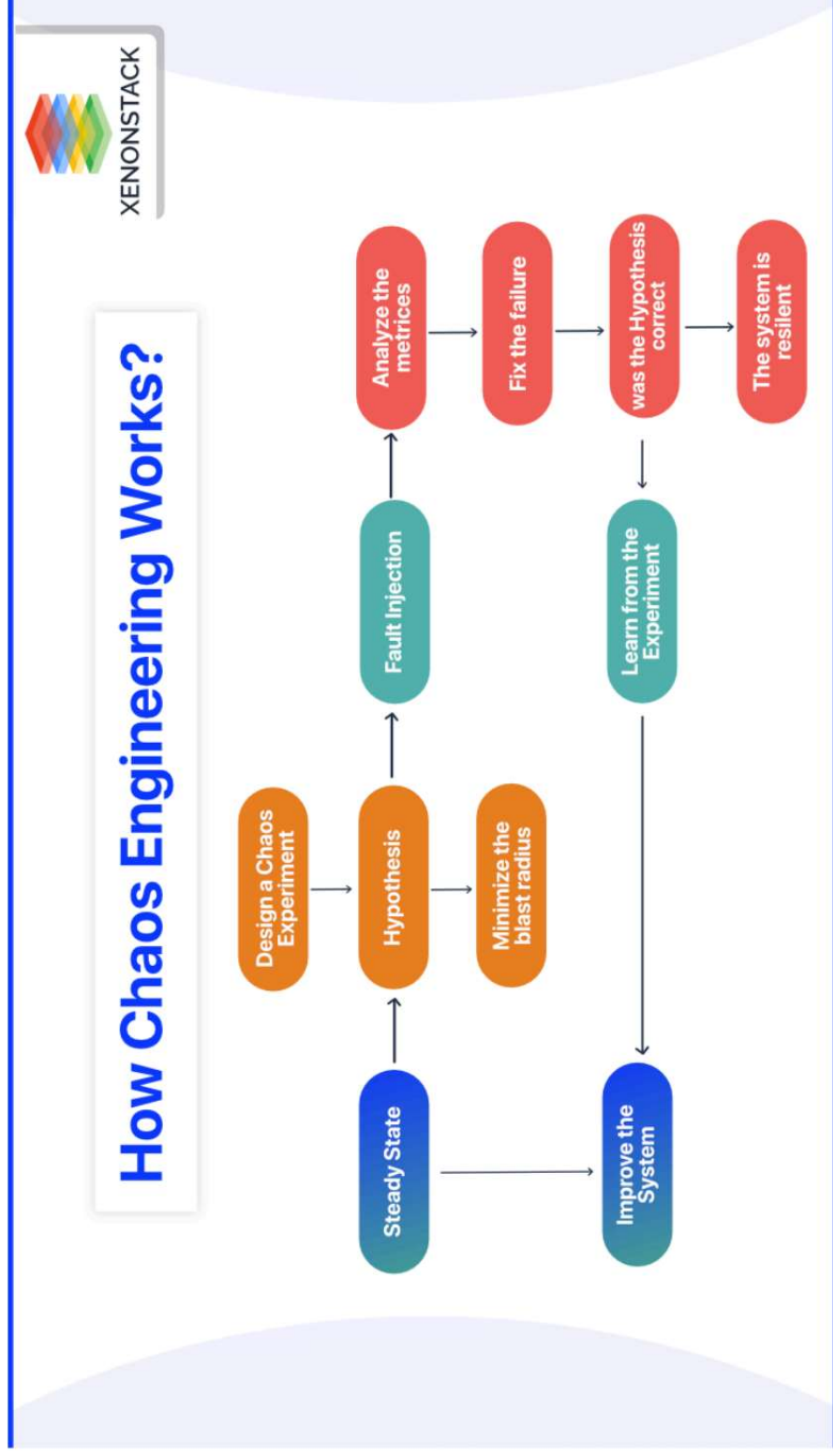


Image Source :<https://www.xenonstack.com/blog/chaos-engineering-for-cloud-native>

Chaos Engineering: Controlled Failure Injection

1. Principles of Chaos Engineering

- Hypothesis-driven testing: Define expectations before injecting failures.
- Gradual expansion of the blast radius: Start with a single node or service and scale.

2. Failure Injection Tools

- Use **Gremlin** or **Chaos Monkey** to simulate failures like:
- Node outages.
- Network latency spikes.
- Disk exhaustion.

3. Monitoring Validation

- Validate observability tools during chaos experiments.
- Ensure systems are resilient and failover mechanisms are effective.

Activity:

Simulate a database failure using **Gremlin** and monitor system behaviour.

Continuous Improvement with Chaos Engineering

1. Structured Chaos Experiments

- Design experiments based on real-world scenarios:
- Simulate peak load on an e-commerce platform during a sale.
- Test auto-scaling mechanisms by reducing available compute resources

2. Learning from Experiments

- Conduct blameless postmortems after each chaos experiment.
- Document outcomes and implement reliability improvements.

Activity:

Run a chaos experiment simulating API rate limiting and discuss mitigation strategies.

Case Study

Case Study: Building Resilient Microservices for a Financial Payment Platform

Scenario Overview

A global financial company, **PaySecure**, provides real-time payment processing for millions of transactions daily. Its platform must deliver **99.99% availability**, handle surges during peak times, and maintain customer trust by ensuring seamless service. However, challenges in observability and system resilience often lead to delayed issue resolution, impacting user satisfaction.

Limited Observability:

- Siloed metrics across infrastructure, application, and business levels.
- No unified system for aggregating logs and traces.
- Difficulty correlating metrics to root causes during incidents.

System Resilience:

- Lack of chaos engineering practices to test system recovery under failure conditions.
- Service degradation during regional database outages or high traffic loads.

Inefficient Alert Management:

- Excessive false-positive alerts lead to alert fatigue among on-call engineers.
- Absence of anomaly detection for proactive issue identification.

Sample Chaos Engineering Experiments

- **Simulate Database Latency:** Inject 200ms latency into the primary database to test the resilience of read replicas and query timeout mechanisms.
- **Terminate Application Instances:** Randomly terminate EC2 instances in an Auto Scaling group to validate scaling policies and recovery time.
- **Introduce Network Partition:** Simulate a network partition between key microservices to test fail-back mechanisms and circuit breaker functionality.
- **Throttle API Gateway:** Apply strict rate limits on API Gateway traffic to evaluate the system's behaviour under throttling scenarios.
- **Inject Pod Failures in Kubernetes:** Kill random pods in a Kubernetes cluster using Chaos Monkey to test pod rescheduling and workload continuity.

Key Takeaways

Observability ensures a system's internal state is transparent for faster debugging and optimization.

Multi-layer monitoring integrates infrastructure, application, and business metrics for holistic insights.

Chaos Engineering systematically enhances resilience through controlled failure injection.

Exercise Case Study



Case Study: Netflix Chaos Engineering

- **Scenario:** Resilience testing during high-traffic streaming events.
- **Implementation:** Using **Chaos Monkey** to terminate instances in production.
- **Outcome:** Improved failover strategies and faster incident resolution.
- **Discussion:**
- How would you design a chaos experiment for your current system?

Observability & Chaos Engineering

Comprehensive Monitoring:

Implement centralized **log aggregation**, **metric collection**, and **distributed tracing** for end-to-end system visibility.

Multi-Layer Monitoring Architecture:

Integrate **infrastructure**, **application**, and **business metrics** to create a holistic monitoring framework for microservices.

Monitoring Best Practices:

Leverage **anomaly detection** and configure **alert management** to reduce noise and ensure efficient issue resolution.

Controlled Failure Injection:

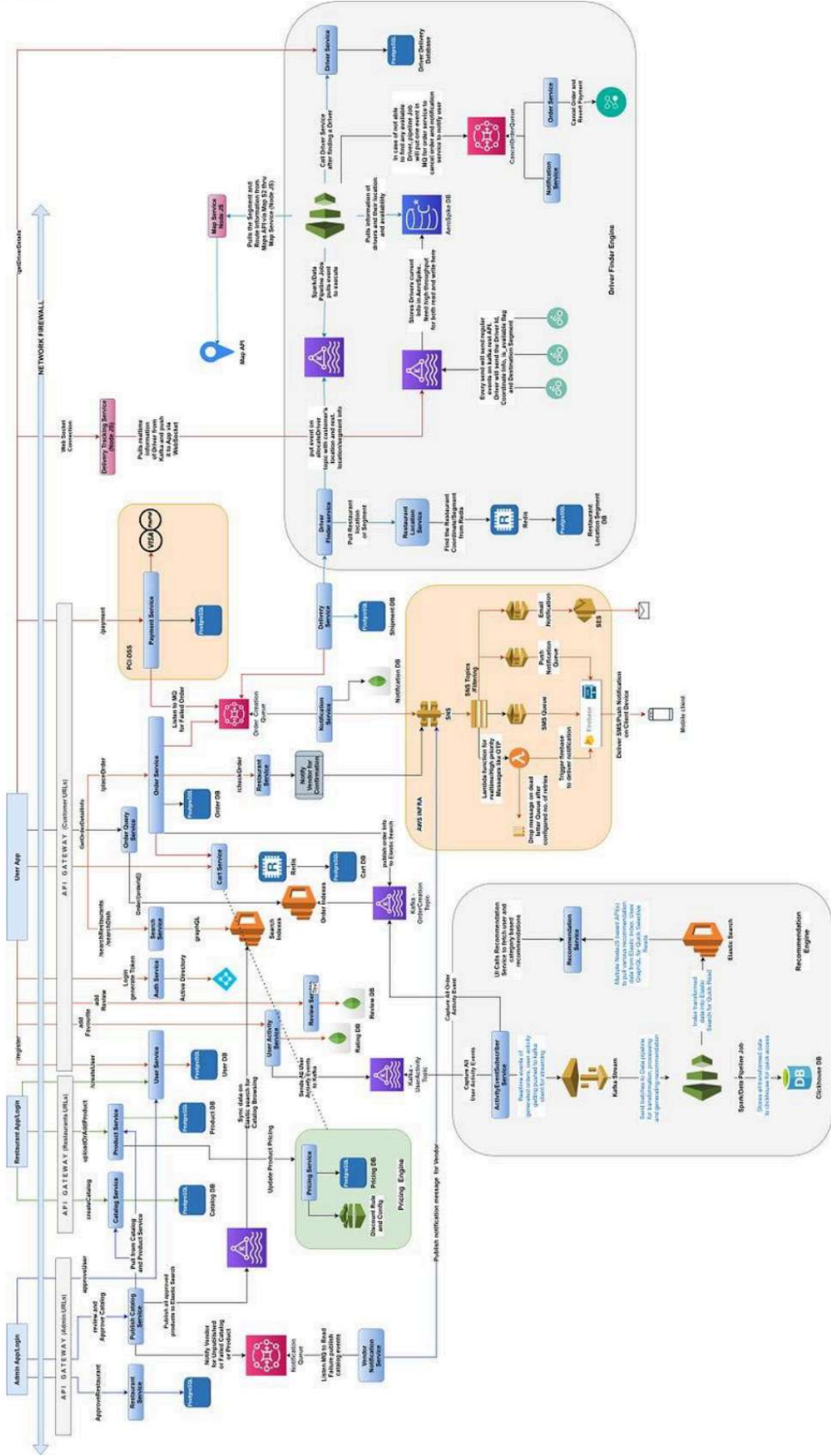
Conduct systematic chaos experiments to validate system resilience and monitor failover mechanisms during simulated failures.

Continuous Improvement:

Use structured chaos engineering to identify vulnerabilities and improve recovery strategies for enhanced reliability and scalability.

Key Takeaway:

Combining **observability** with **chaos engineering** builds proactive, resilient, and scalable systems that meet modern IT demands.



Thank You!
StackRoute
(An NIIT Venture)

